# L06: CNN Architectures & Transfer Learning

FICT, UTAR

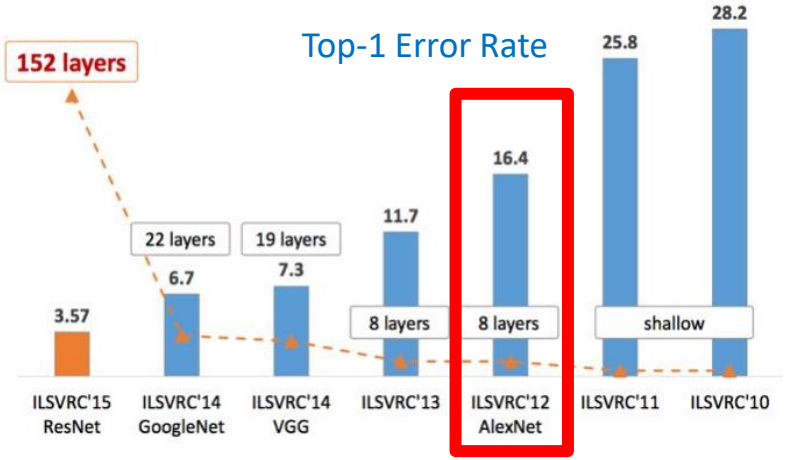
# Table of Contents

# Network Architecture:
# **AlexNet**

# AlexNet (2012)

[Krizhevsky, A., Sutskever, I. and Hinton, G.E., Imagenet classification with deep convolutional neural networks. NIPS2012]

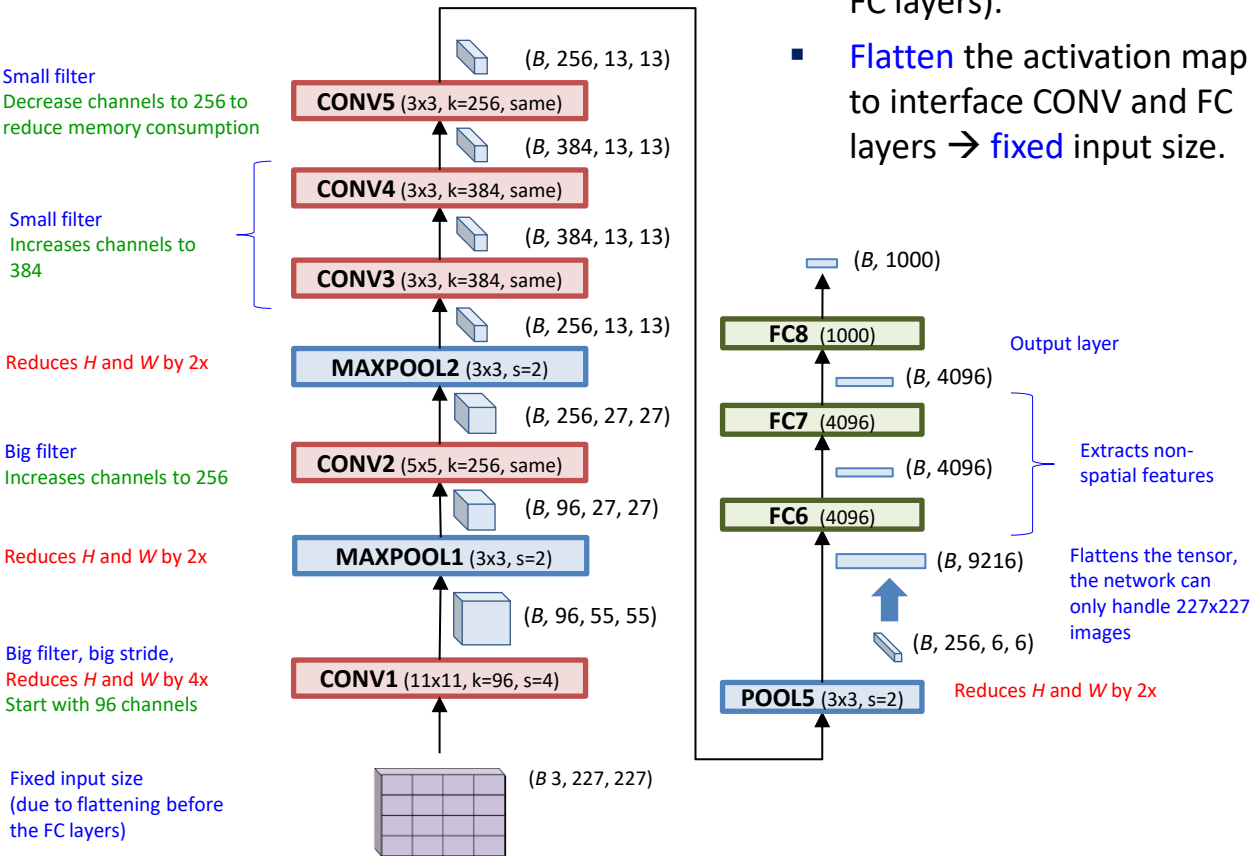
- Winner of ILSVRC-2012 competition for object classification & localiza




- Jaw-dropping performance (top-1 error: 16.4%, top-5 error 15.3%)
- Beats all hand-crafted method (2nd best: top-5 error: 26.2%)
- Triggered the deep learning revolution in Computer Vision
- Utilizes many techniques that are still used today

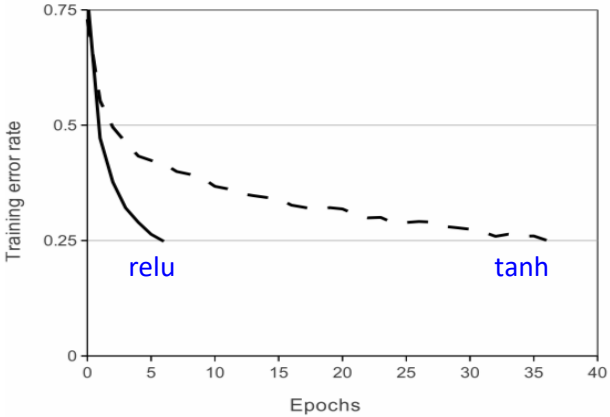
# AlexNet Architecture

- Total: 8 layers (5 CONV + 3 FC layers).
- Flatten the activation map to interface CONV and FC layers → fixed input size.



# Useful Practices from AlexNet

- First use of ReLU activation (6 times faster than tanh)




- Training
  - Optimizer: Stochastic Gradient Descent with momentum ($\beta$=0.9)
  - Multiple GPUs to train efficiently (#gpus = 2)
  - Learning rate decay (set $\alpha_0$ = 0.01, divide by 10 when validation stops improving, 3x prior to termination)

# Useful Practices from AlexNet

- Regularization:
  - Applies dropout after CONV1 and CONV2 ($p$=0.5) – reduces overfitting but double # epoch to converge
  - Data augmentation to increase the training set by a factor of 2048 (random crop, random flip, color jitter)
  - Weight decay or L2 regularization (weight decay = 0.0005)
- Inference:
  - 10-crop testing (mean inference result on 10 cropped locations)
  - 7 CNN Ensemble reduces error from 18.2% to 15.4%

# Issues with AlexNet

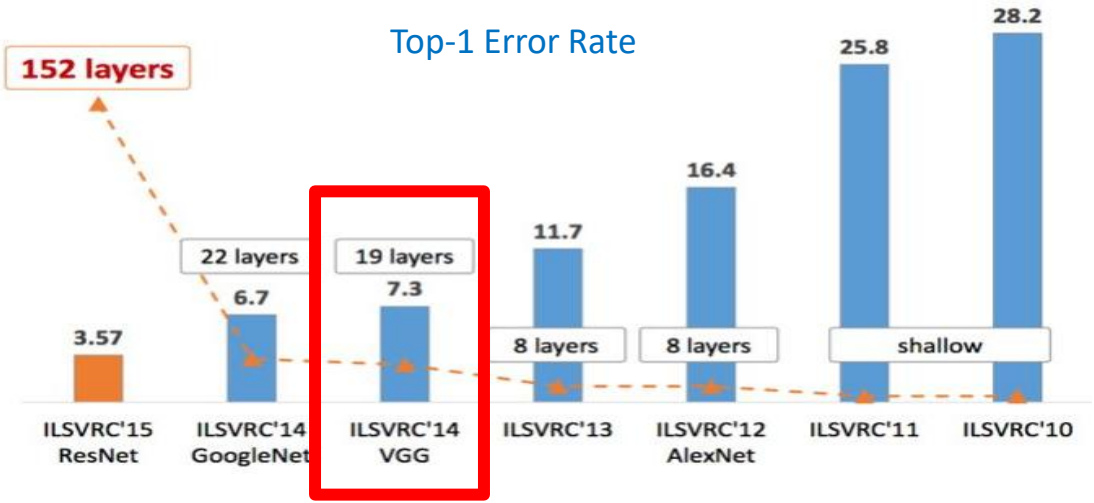
- Not deep: 8 layers only.
  - Use of big filters and strides.
  - Less powerful by today's standard.
  - Huge number of parameters.
- Design is not regular.
  - Configuration of the layers varyies in each layer (different *f* and *s*).
  - Size of the activation maps varies in each layer.

# Network Architecture:
# **VGGNet**

# VGGNet (2015)

Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. 3rd International Conference on Learning Representations (ICLR 2015), 1–14.

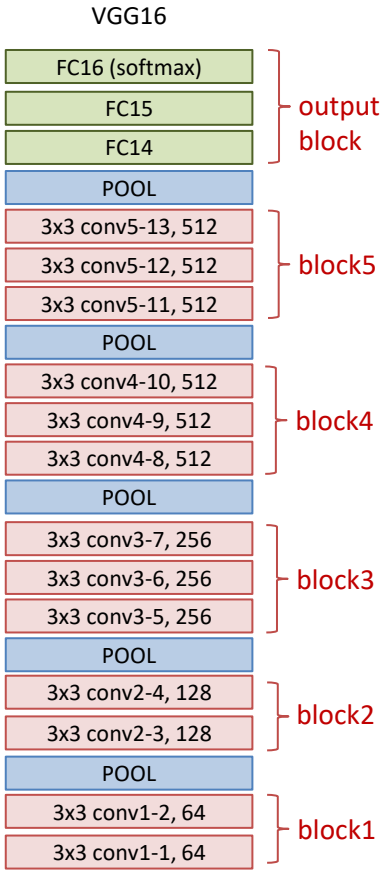
- ILSVRC 2014: Runner-up for classification task in ILSVRC2014



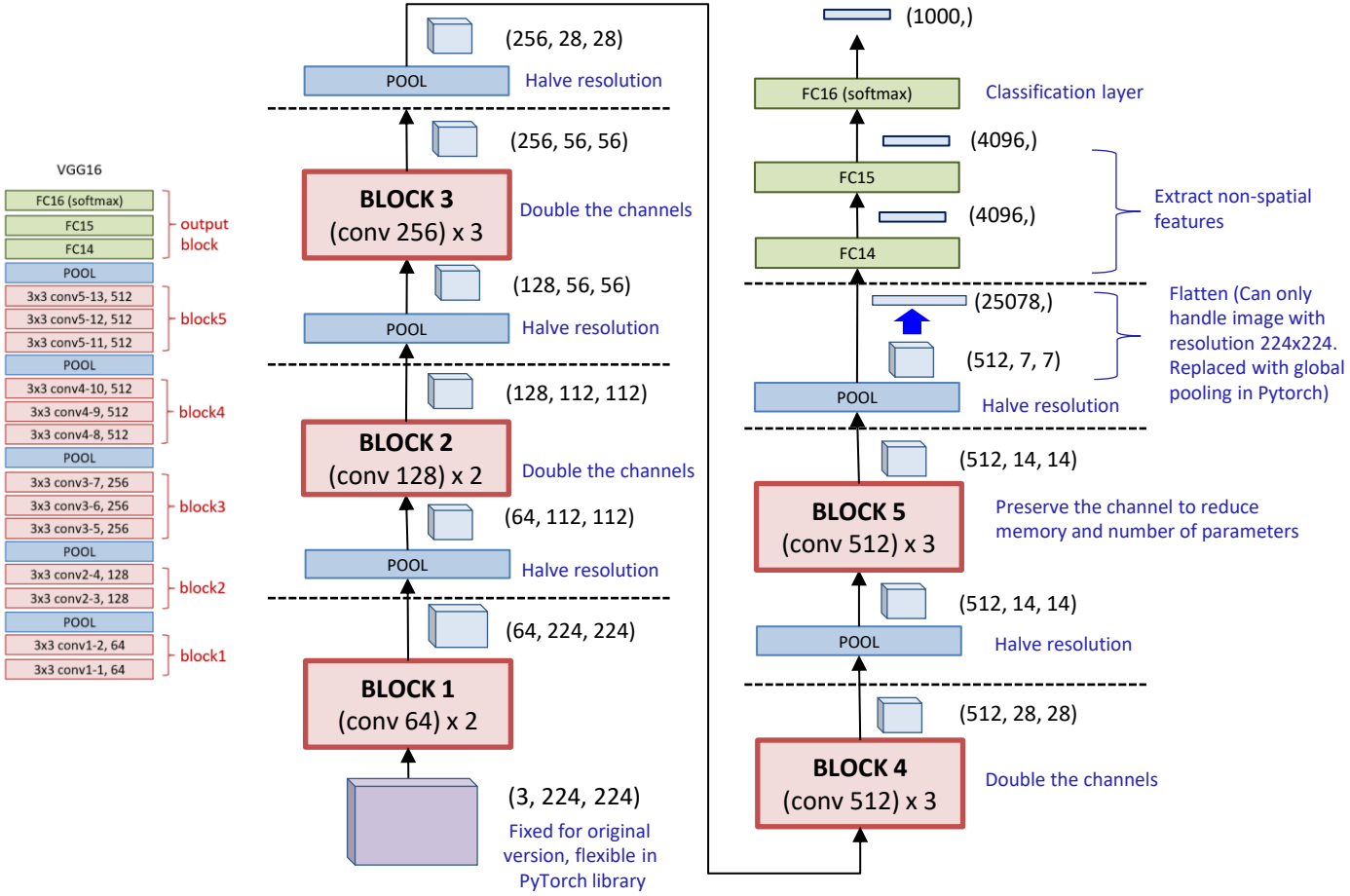

Top-5 error rate of 5.1% (Recall AlexNet: 15.3%)

# Design Philosophies of VGGNet

- Deeper network (compared to AlexNet)
  - 16 and 19 layers
- Regular and principled design:
  - ✓ Only small filter (3x3)
  - ✓ Layers are organized into homogeneous blocks (activation map has same resolution)
  - ✓ Use pooling (not conv) to reduce spatial dimension
  - ✓ When the resolution is reduced by half, number of channels is doubled



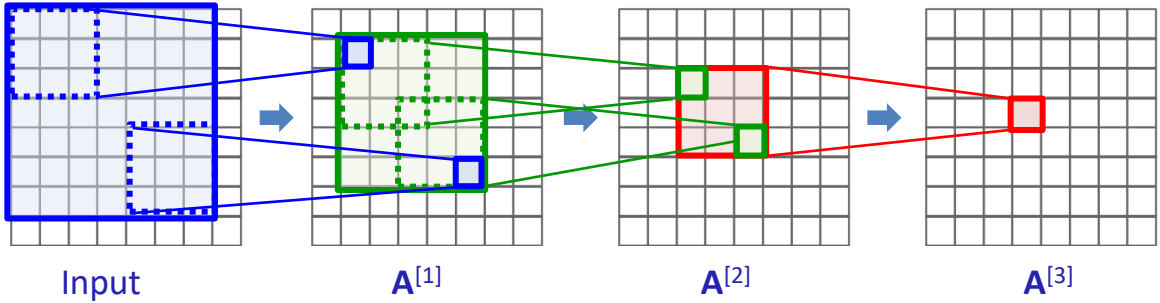
# VGG-16 Architecture



# Receptive Field

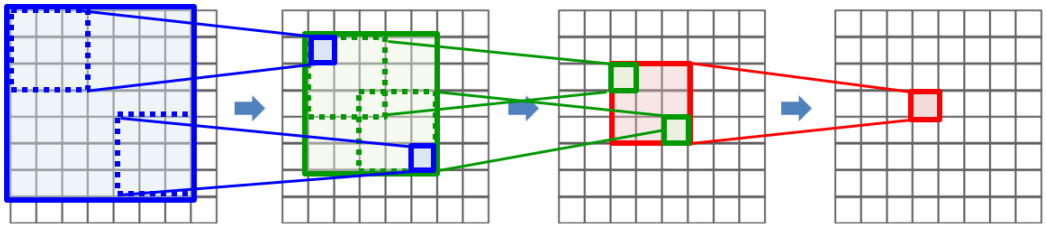
- 3x3 filters is small. Can it detect patterns at larger scales?

  Yes. The receptive field increases with depth. Receptive field refers to the resolution of the region of a lower layer that is used to compute an activation value at a particular layer.

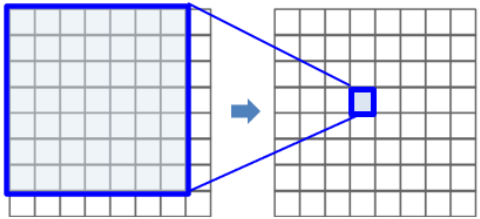


# Filter size and depth

- Smaller filters can achieve the same receptive field as a big filter by having more layers:
- Example:
  - Conv 3x3 needs 3 layers to achieve a receptive field of 7x7 at the input layer.

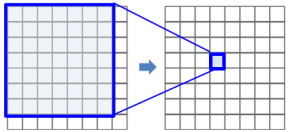


  - Conv 7x7 needs 1 layers to achieve a receptive field of 7x7.

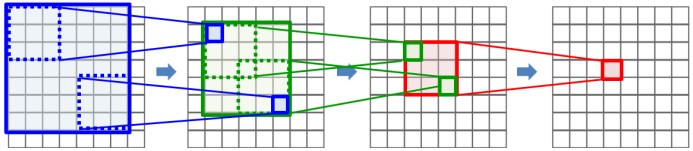


# Slimmer but deeper network

- Deeper networks have more activation functions which enables it to model more complex functions.



1 ReLU activation



3 ReLU activations

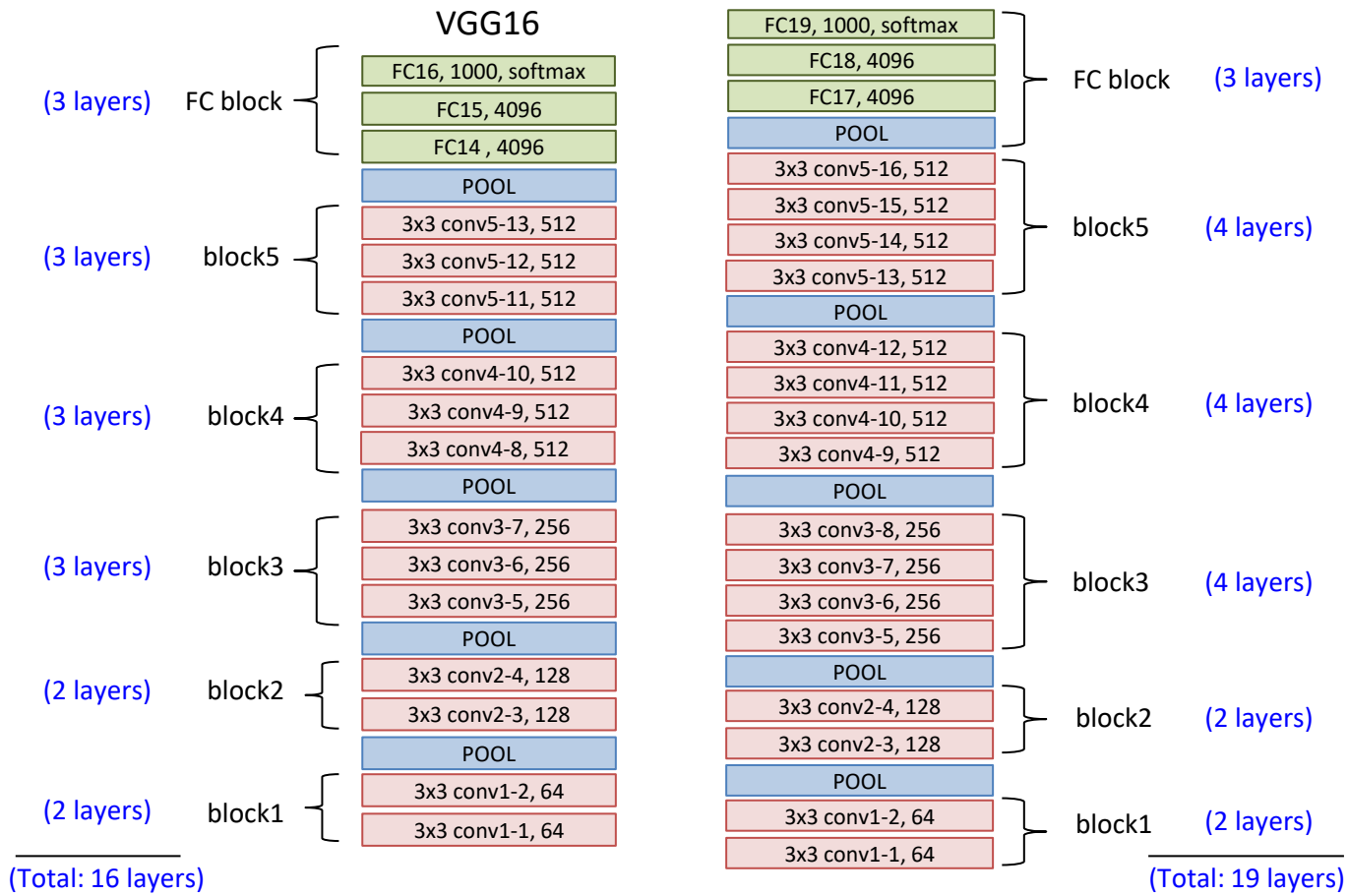
- Less #parameters, hence easier to train.

Assume that all layers have $c_{out}$ filters and $c_{in}$ input channels:

$\#\text{param}_{f=7}$
$= (7\times7\times c_{in}+1) \times c_{out} \times 1$ layer
$= 49 c_{out} c_{in} + k$

$\#\text{param}_{f=3}$
$= (3\times3\times c_{in}+1)\times c_{out} \times 3$ layers
$= 27 c_{out} c_{in} + 3k$
(Only 55% of larger filter)

# Variants of VGGNet



# #Parameters and memory consumption

| | | |
|---|---|---|
| INPUT: (3,224,224) | memory: 224*224*3=150K | weights : 0 |
| CONV3-64: (64,224,224) | memory: 224*224*64= 3.2M | weights: (3*3*3)*64 = 1,728 |
| CONV3-64: (64,224,224) | memory: 224*224*64= 3.2M | weights : (3*3*64)*64 = 36,864 |
| POOL2: (64,112,112) | memory: 112*112*64=800K | weights : 0 |
| CONV3-128: (128,112,112) | memory: 112*112*128=1.6M | weights : (3*3*64)*128 = 73,728 |
| CONV3-128: (128,112,112) | memory: 112*112*128=1.6M | weights : (3*3*128)*128 = 147,456 |
| POOL2: (128,56,56) | memory: 56*56*128=400K | weights : 0 |
| CONV3-256: (256,56,56) | memory: 56*56*256=800K | weights : (3*3*128)*256 = 294,912 |
| CONV3-256: (256,56,56) | memory: 56*56*256=800K | weights : (3*3*256)*256 = 589,824 |
| CONV3-256: (256,56,56) | memory: 56*56*256=800K | weights : (3*3*256)*256 = 589,824 |
| POOL2: (256,28,28) | memory: 28*28*256=200K | weights : 0 |
| CONV3-512: (512,28,28) | memory: 28*28*512=400K | weights : (3*3*256)*512 = 1,179,648 |
| CONV3-512: (512,28,28) | memory: 28*28*512=400K | weights : (3*3*512)*512 = 2,359,296 |
| CONV3-512: (512,28,28) | memory: 28*28*512=400K | weights : (3*3*512)*512 = 2,359,296 |
| POOL2: (512,14,14) | memory: 14*14*512=100K | weights : 0 |
| CONV3-512: (512,14,14) | memory: 14*14*512=100K | weights : (3*3*512)*512 = 2,359,296 |
| CONV3-512: (512,14,14) | memory: 14*14*512=100K | weights : (3*3*512)*512 = 2,359,296 |
| CONV3-512: (512,14,14) | memory: 14*14*512=100K | weights : (3*3*512)*512 = 2,359,296 |
| POOL2: (512,7,7) | memory: 7*7*512=25K | weights : 0 |
| FC: (4096,) | memory: 4096 | weights : 7*7*512*4096 = 102,760,448 |
| FC: (4096,) | memory: 4096 | weights : 4096*4096 = 16,777,216 |
| FC: (1000,) | memory: 1000 | weights : 4096*1000 = 4,096,000 |

TOTAL memory: 24M * 4 bytes ~= 96MB / image (only forward! ~*2 for bwd)

TOTAL weights: 138M

Most memory consumption at the early CONV layers

Most params are at the FC layer(difficult to train). PyTorch replaces this with global pooling

Very heavy memory usage
~0.2 GB for each image. For a GPU with 8GB, can only support up to a batch size of 40 images

Many parameters (AlexNet only has 60M)

# Weakness of VGGNet

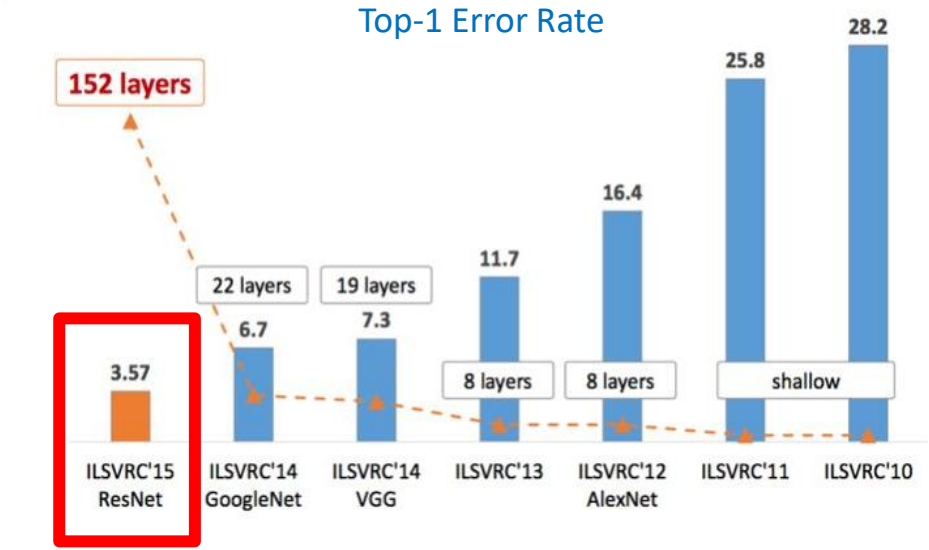
- Slow to train
- Huge number of parameters to train
- High memory consumption

# Network Architecture:
# **ResNet**

# Residual Network (ResNet)

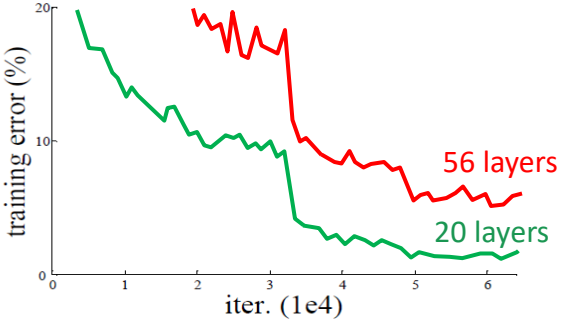
He, K., Zhang, X., Ren, S. and Sun, J., 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770-778).

- Winner for classification, localization and detection tasks (all tasks) in ILSVRC2015

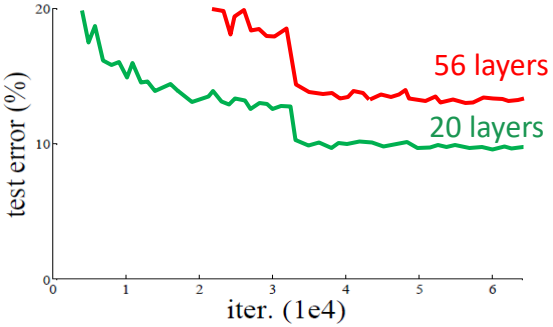


# Motivation for Residual Network

- What is the issue with the result below?




Training error on CIFAR-10

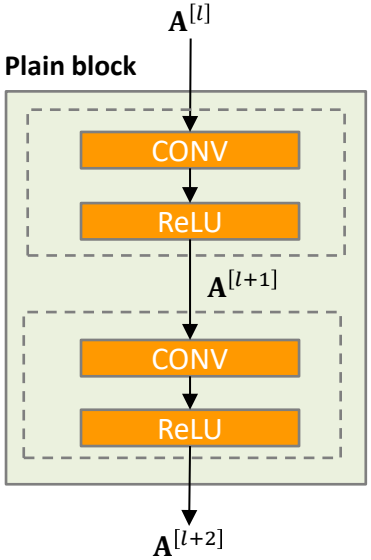



Testing error on CIFAR-10

- Both higher training and testing error for the deeper network → Not caused by overfitting (if yes, the deeper network would have a lower training error)
- High training and test error → difficulty training deeper networks caused by vanishing gradient effect

# Plain Block is learning a mapping function (hard)

- The plain block (two layers) is learning a mapping function




$$\mathbf{Z}^{[l+1]} = \mathbf{W}^{[l+1]}\mathbf{A}^{[l]} + \mathbf{b}^{[l+1]T}$$

$$\mathbf{A}^{[l+1]} = g\left(\mathbf{Z}^{[l+1]}\right)$$

$$\mathbf{Z}^{[l+2]} = \mathbf{W}^{[l+2]}\mathbf{A}^{[l+1]} + \mathbf{b}^{[l+2]T}$$

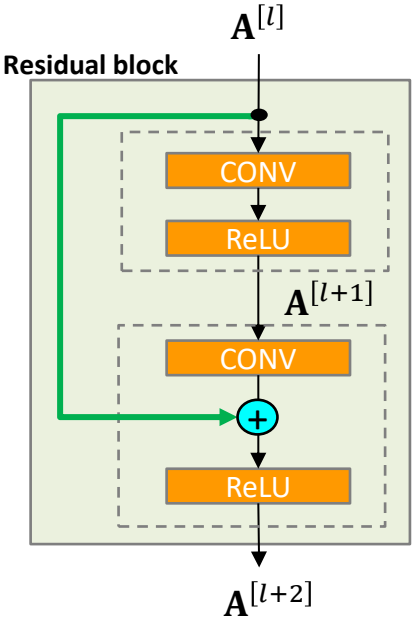
$$\mathbf{A}^{[l+2]} = g\left(\mathbf{Z}^{[l+2]}\right)$$

Learns the mapping $f: \mathbf{A}^{[l]} \rightarrow \mathbf{A}^{[l+2]}$

- Difficult for a mapping function to learn certain functions, e.g., the identity function $\mathbf{A}^{[l+2]} = f\left(\mathbf{A}^{[l]}\right) = \mathbf{A}^{[l]}$

# Residual Block

- Residual block = plain block with short-cut or skip connection.
- Learn a residual function instead of a mapping function.
- The skip connection allows the network to pass earlier information deeper into the layer.




$$\mathbf{Z}^{[l+1]} = \mathbf{W}^{[l+1]}\mathbf{A}^{[l]} + \mathbf{b}^{[l+1]^T}$$

$$\mathbf{A}^{[l+1]} = g\left(\mathbf{Z}^{[l+1]}\right)$$

$$\mathbf{Z}^{[l+2]} = \mathbf{W}^{[l+2]}\mathbf{A}^{[l+1]} + \mathbf{b}^{[l+2]^T}$$

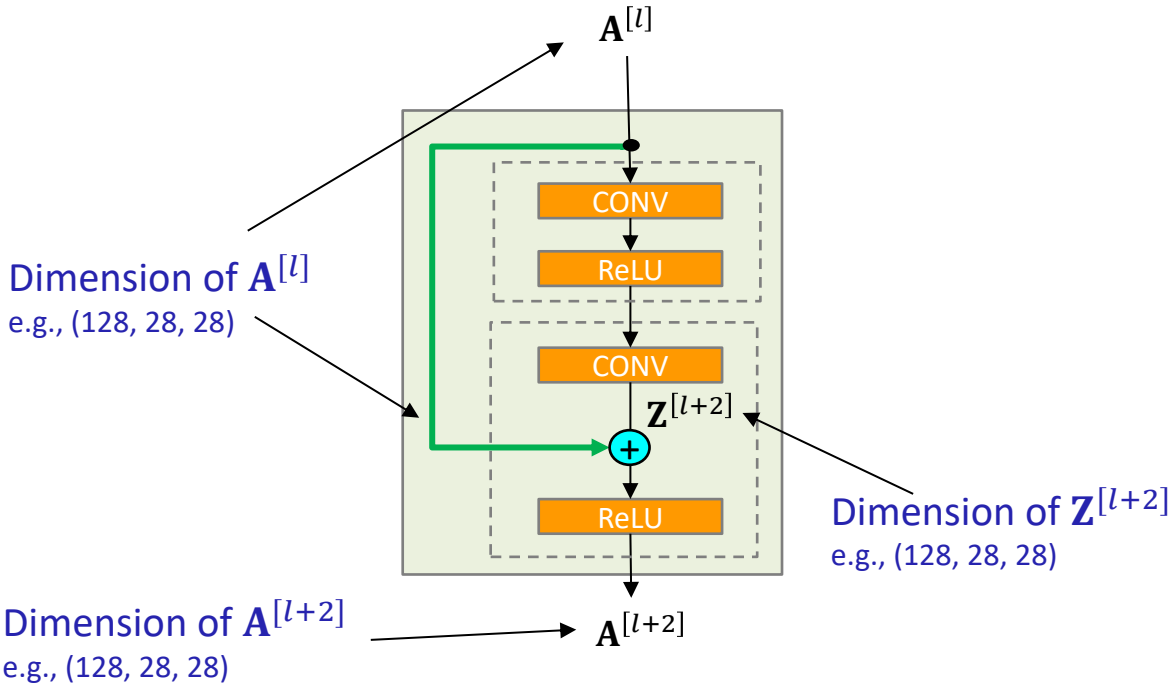
Learns the offset or residual $\mathbf{Z}^{[l+2]} = r\left(\mathbf{A}^{[l]}\right)$

$$\mathbf{A}^{[l+2]} = g\left(\mathbf{Z}^{[l+2]} + \mathbf{A}^{[l]}\right)$$

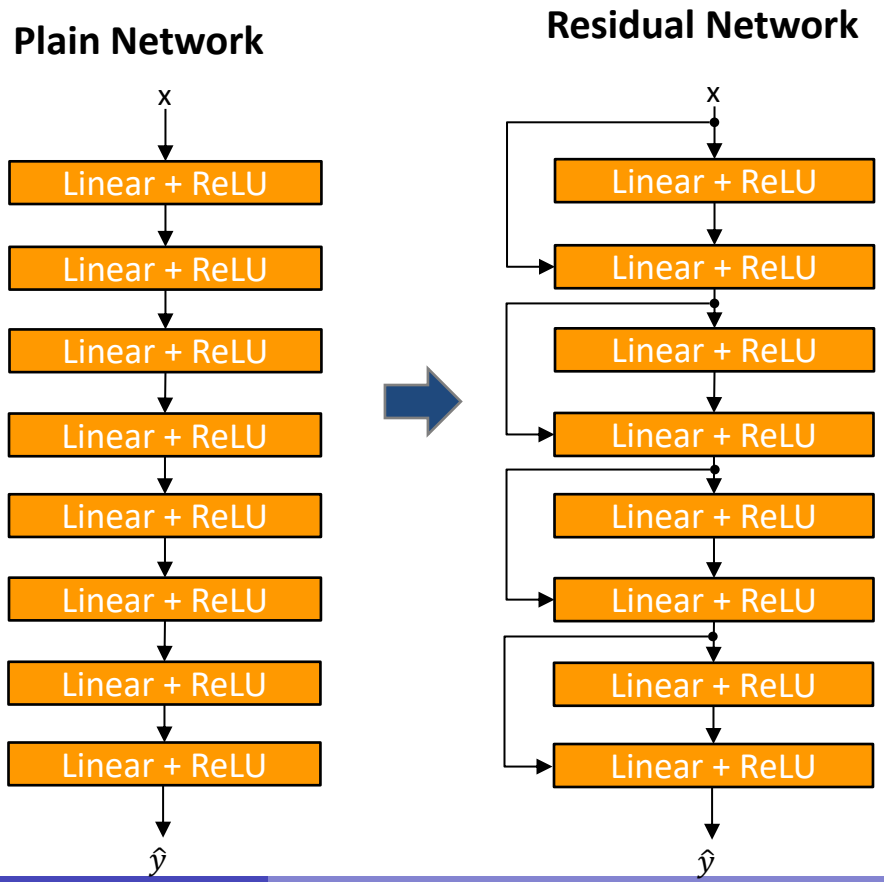
Pre-activation is computed by adding the computed offset $\mathbf{Z}^{[l+2]} = r\left(\mathbf{A}^{[l]}\right)$

# Dimension of the Residual Block

- The inputs to the residual add function $\mathbf{A}^{[l]}$ and $\mathbf{Z}^{[l+2]}$ must have the same dimension.



# Transforming Plain Network to Residual Network

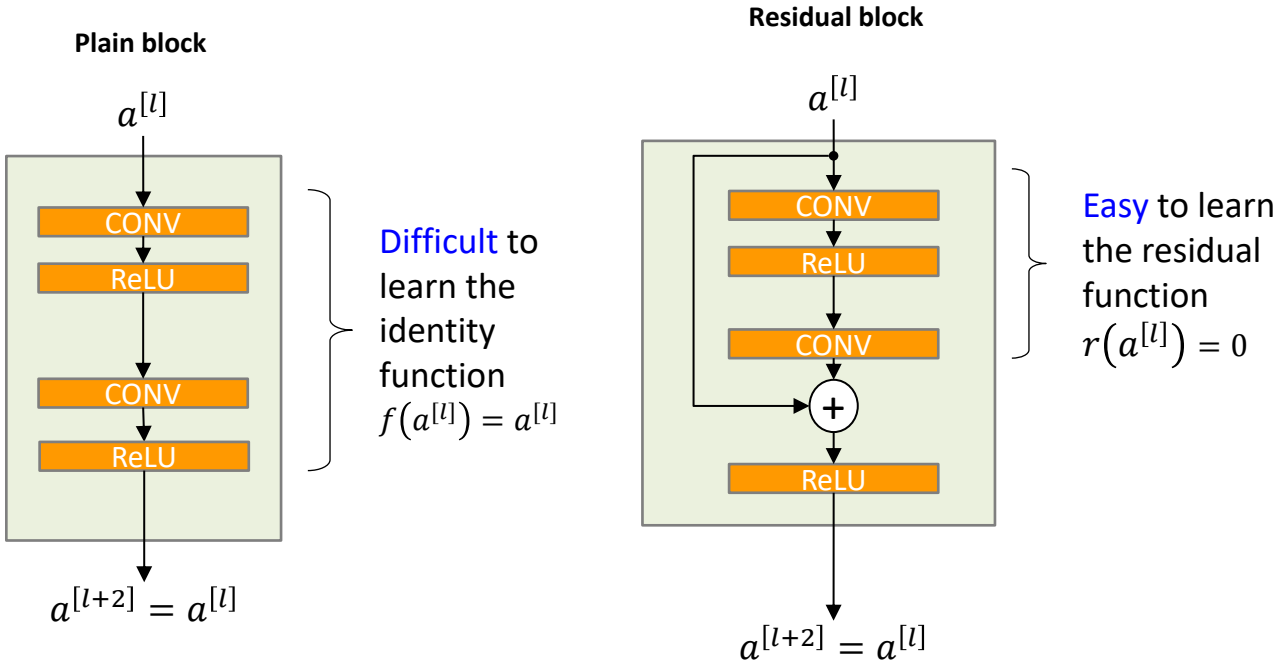



Residual network is built by adding skip connections onto a Plain Network

# Why Residual Block?

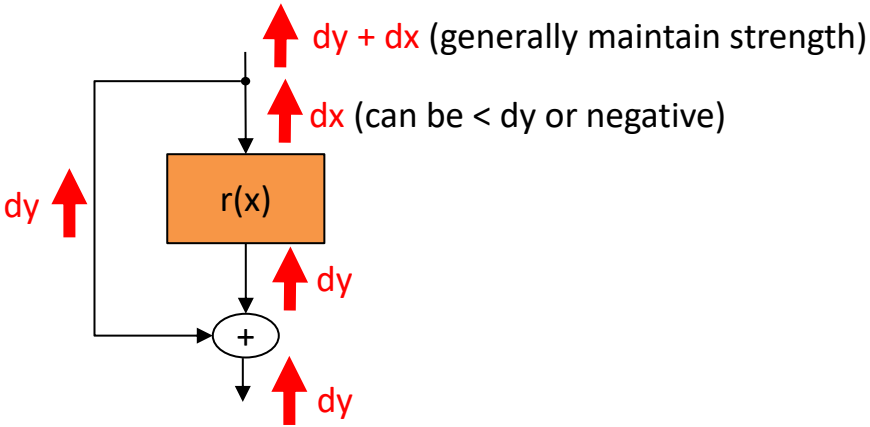
- Easier to learn a residual function than a mapping function

Example: let's say we want to learn an identity function $a^{[l+2]} = a^{[l]}$



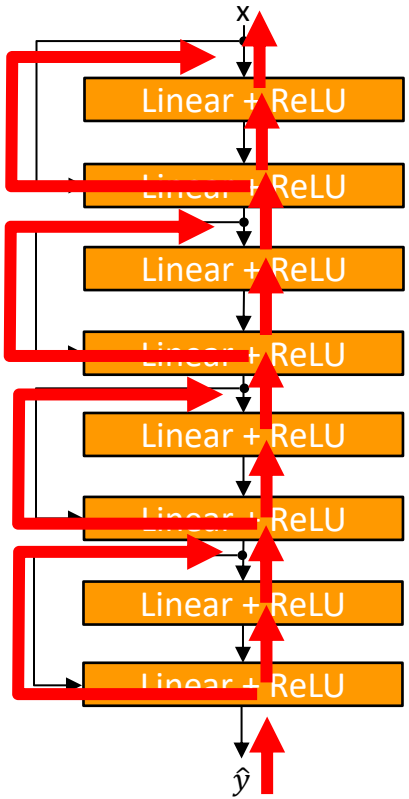
# Why Residual Block?

- Solves the vanishing gradient issue. The short-cut connections provide an uninterrupted path for backpropagating the gradient




add operation is a gradient distributor
branch is a gradient accumulator

**Residual Network**



# Output Layer Design

- Recap: AlexNet and VGGNet use flattening layer and 3 FC layers.
- Issues with using flattening layer:

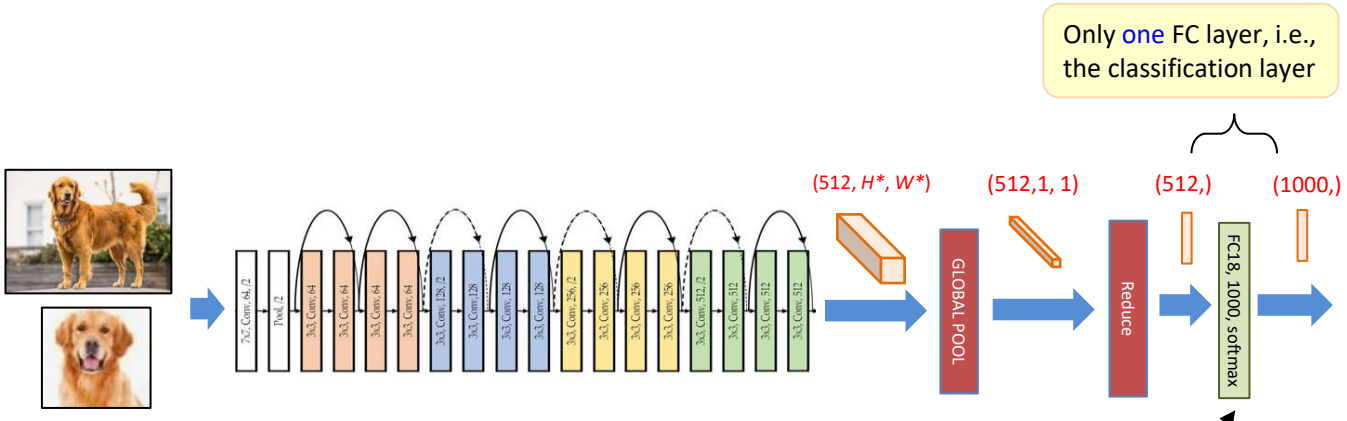
**Issue 2**: Is it really necessary to have multiple FC layers? Not really!


**Issue 3**: The feature to FC14 is fixed at 25088 dimensions → Input image must have a shape of (3, 224, 224)

**Issue 1**: Flattening results in high dimensional vector → huge number of parameters in FC14 (102,760,448 or ~74% of all parameters)

# Output Layer Design

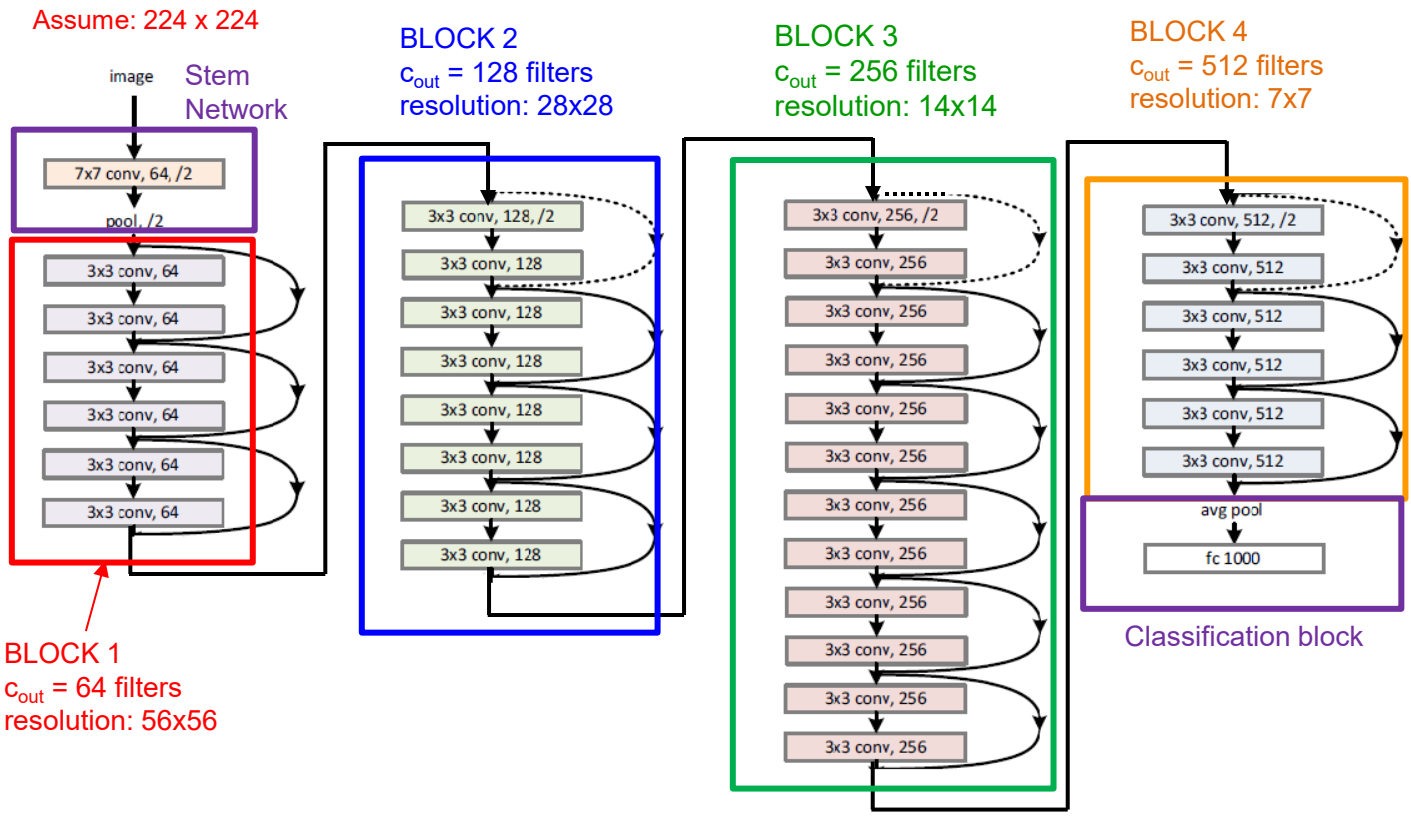
- ResNet uses global pooling as the interface between CONV and FC layers.
- Only one fully connected layer in the output block.



# Design Principles of ResNet

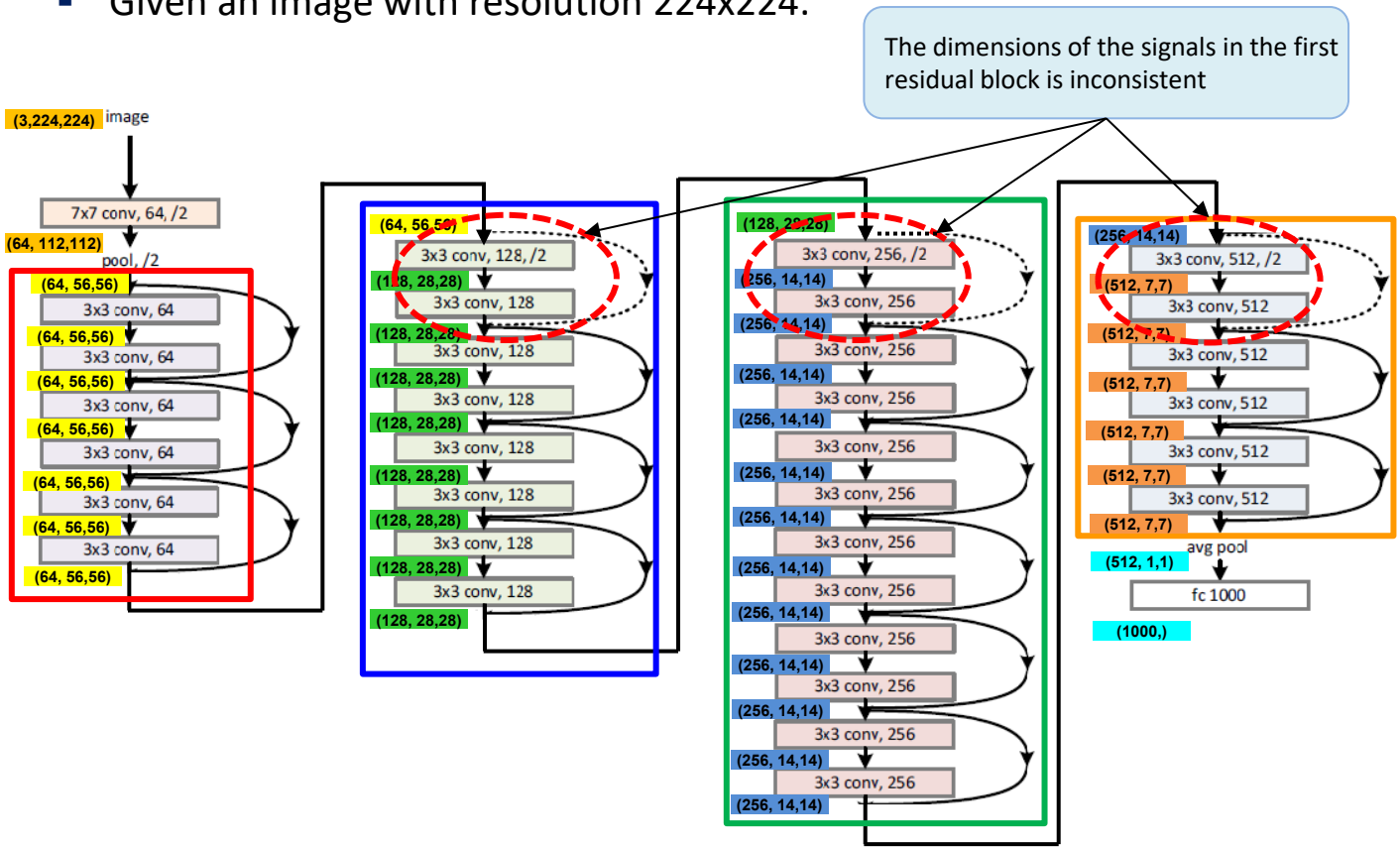
- Adopts the homogeneous design principle of VGG.
  - Mostly have 3x3 filters.
  - Layers within the same block has the same configuration and resolution.
  - When resolution is halved, number of channels is doubled to preserve the complexity per layer.
- Downsampling is performed directly by convolutional layers with a stride of 2.
- Efficient output block design
  - Uses global average pooling to interface between the CONV and FC blocks
  - Only one FC layer
  - Small number of parameters and memory consumption
  - Less overfitting issues

# ResNet Architecture (ResNet34)



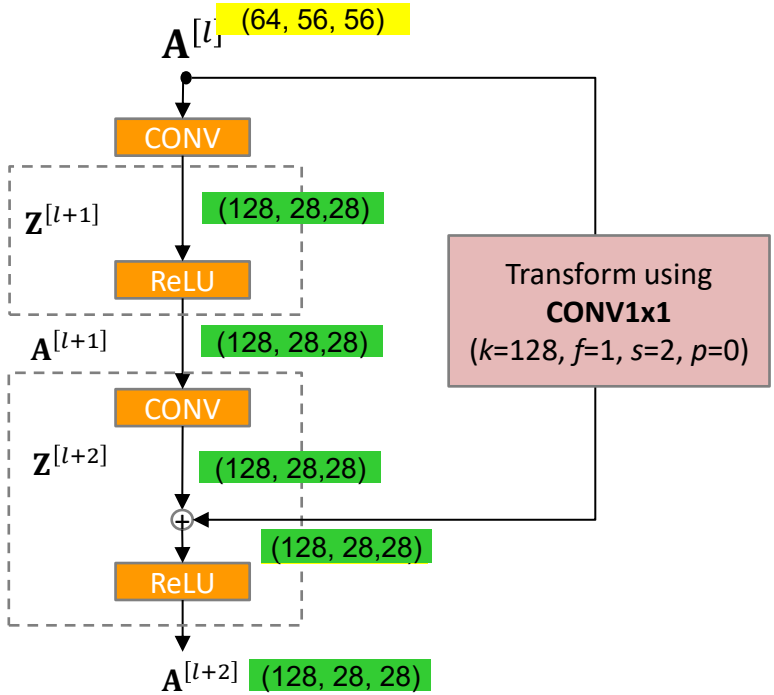
# Dimensions of activation maps and features

- Given an image with resolution 224x224:



# Design of the First Residual Block

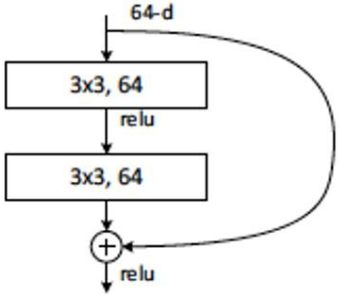
- If the depth of $\mathbf{A}^{[l]}$ and $\mathbf{Z}^{[l+2]}$ are different, perform 1x1 convolution with stride = 2 and $c_{out}$= 2*$c_{in}$.



# Basic vs Bottleneck Residual Block

- To reduce computation time and number of parameters, use a bottleneck design to design the building block for deeper versions of ResNet (50, 101 and 152).

**Basic building block**
(Used in ResNet-18, 34)

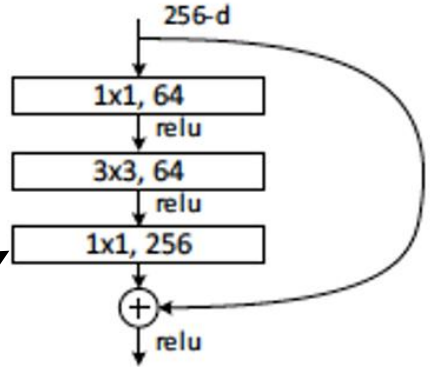



**Bottleneck building block**
(Used in ResNet-50, 101, 152)

CONV1x1 shrinks the channels from 256 to 64 to reduce the input activation map for 3x3

CONV 3x3 extracts feature on a smaller input with 64 rather than 256 channels (more efficient)

CONV1x1 expands the channels back to 256 to preserve the input dimension



# Variants of ResNet

Assume input resolution is (224, 224)

| layer name | output size | 18-layer | 34-layer | 50-layer | 101-layer | 152-layer |
|---|---|---|---|---|---|---|
| conv1 | 112×112 | 7×7, 64, stride 2 | | | | |
| | | 3×3 max pool, stride 2 | | | | |
| conv2_x | 56×56 | $\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$ | $\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$ |
| conv3_x | 28×28 | $\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$ | $\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$ | $\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$ | $\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$ | $\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$ |
| conv4_x | 14×14 | $\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$ | $\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$ | $\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$ | $\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$ | $\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$ |
| conv5_x | 7×7 | $\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$ | $\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$ |
| | 1×1 | average pool, 1000-d fc, softmax | | | | |
| FLOPs | | $1.8\times10^9$ | $3.6\times10^9$ | $3.8\times10^9$ | $7.6\times10^9$ | $11.3\times10^9$ |

Uses Basic Residual Block

Uses Bottleneck Residual Block

# Experimental Results

- Training ResNet:
  - Batch Normalization after every CONV layer
  - Xavier initialization from He et al.
  - SGD + Momentum (0.9)
  - Learning rate = 0.1, divided by 10 when validation error plateaus
  - Mini-batch size o256
  - Weight decay of 1e-5
  - No dropout used

- Experimental results
  - Able to train very deep network without degradation (152 layers on ImageNet, 1202 layers on CIFAR10)
  - Deeper networks can achieve lower training error as expected
  - 1$^{st}$ place in all ILSVRC and COCO 2015 competitions

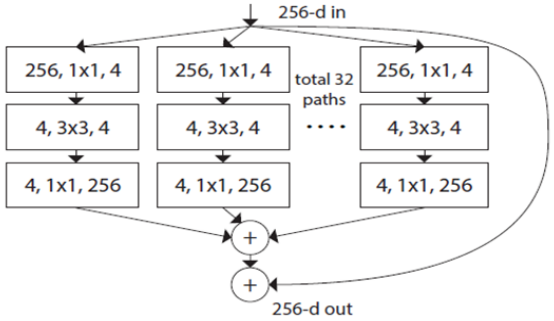
# Network Architecture:
# **ResNeXt**

# ResNeXt (2017)

Xie, S., Girshick, R., Dollár, P., Tu, Z. and He, K., 2017. Aggregated residual transformations for deep neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 1492-1500).

- Similar overall structure as ResNet but uses ResNeXt layers which processes the input in different paths or groups.
  - Each path has fewer channels (e.g., 4) → more efficient.
  - Each path learns different types of features independently → more diverse representations and better generalization
- No of paths = cardinality of the network. 32x4d means cardinality=32, channels=4

A ResNeXt block (32x4d)




**Operations of ResNeXt Block**

1. Same input goes to multiple paths
2. For each path:
   - Shrink channels with CONV1x1
   - Perform CONV3x3 (efficient since channels have been reduced)
   - Revive channels with CONV1x1
3. Merge paths by adding activations from all paths

- Often outperforms ResNet on large-scale datasets, e.g., ImageNet. But not necessarily so on small-scale datasets or certain tasks.

# ResNeXt (2017)

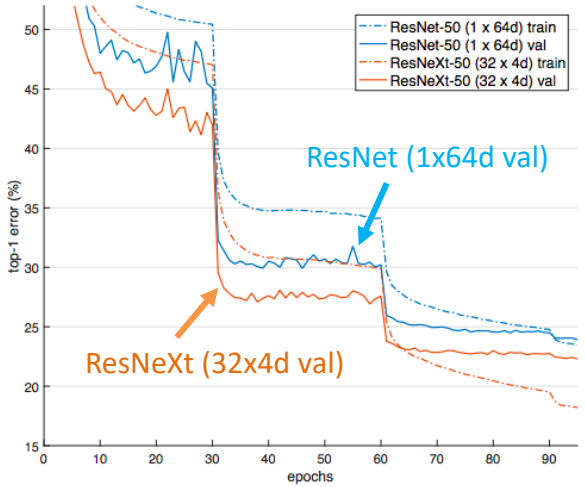
Why having multiple groups work better?

- **Independent learning path** – capture different aspects of the data more efficiently. Prevent issues such as co-adaptation.
- **Enhanced expressive power** –splitting convolution into multiple groups and combining the outputs allows the network to learn a richer and more diverse set of representation.
- **Efficient Use of Parameters** – increase network capacity without increasing the computational cost linearly.
- **Improved gradient flow** – gradients from each group propagate more independently.

# ResNeXt (2017)

- Replacing ResNet layers with ResNeXt layers often improves the performance of the model.

Depth = 50




Depth = 101

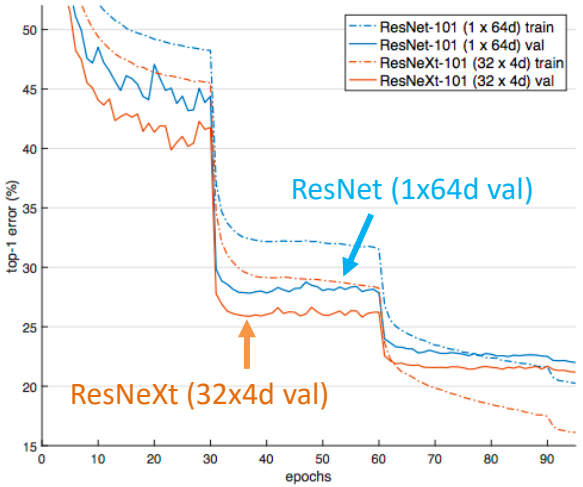


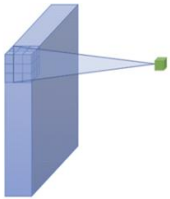

Figure 5. Training curves on ImageNet-1K. (**Left**): ResNet/ResNeXt-50 with preserved complexity (~4.1 billion FLOPs, ~25 million parameters); (**Right**): ResNet/ResNeXt-101 with preserved complexity (~7.8 billion FLOPs, ~44 million parameters).

# Network Architecture:
# **MobileNet**

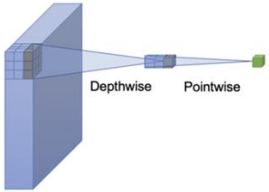
# MobileNet

(Howard, Andrew G., et al., Mobilenets: Efficient convolutional neural networks for mobile vision applications. 2017)

- Efficient deep learning model for mobile and edge devices
- Replaces standard convolutions with **Depthwise Separable Convolution** → fewer parameters, lower computation and comparable accuracy

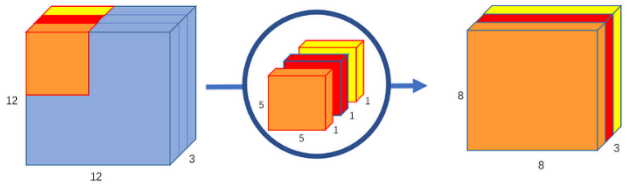


Standard Convolution

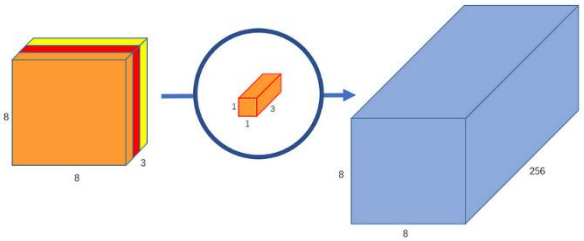



Depth-wise Separable Convolution

- Depthwise Separable Convolution splits spatial filtering and feature combination:

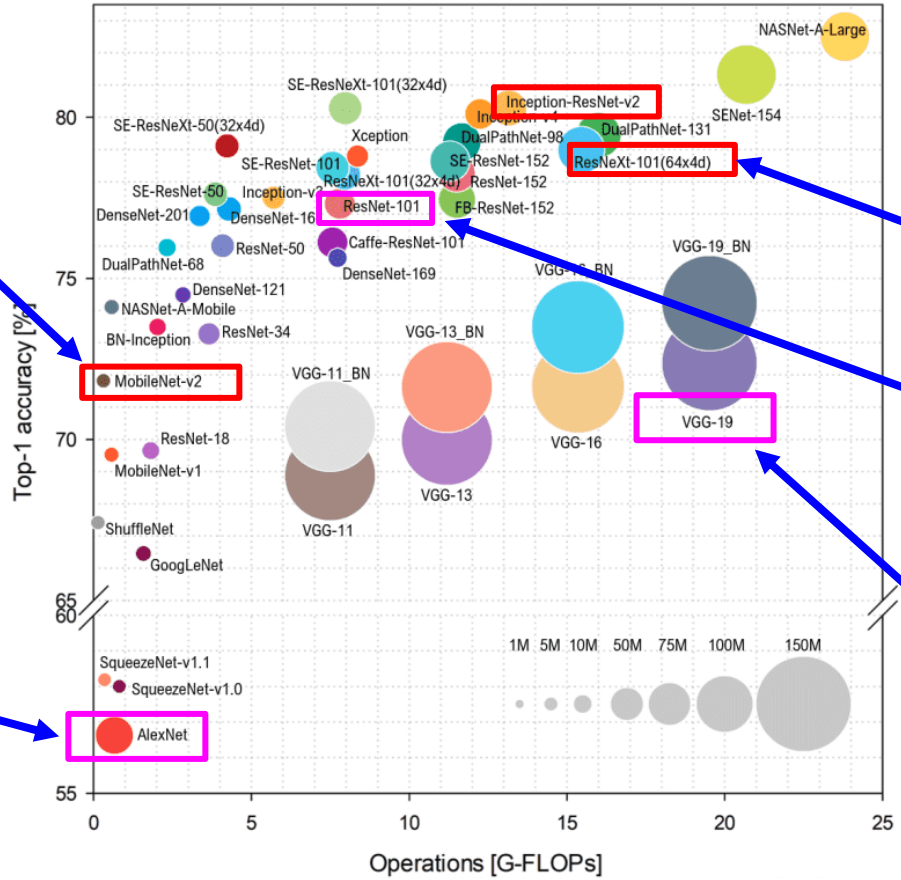



**Depth-wise convolution**: Applies a single filter per input channel with $c_f = 1$ to capture spatial features.




**Point-wise convolution**: Uses 1x1 convolutions with $c_f = c_{in}$ to combine channel information.

# Performance of CNN Architectures




MobileNet: Very efficient and small

ResNeXt, slower than ResNet, but higher accuracy

ResNet: Good accuracy, not too big, not too slow

VGG: Highest memory consumption, slow

AlexNet: smaller compute, memory heavy, low accuracy

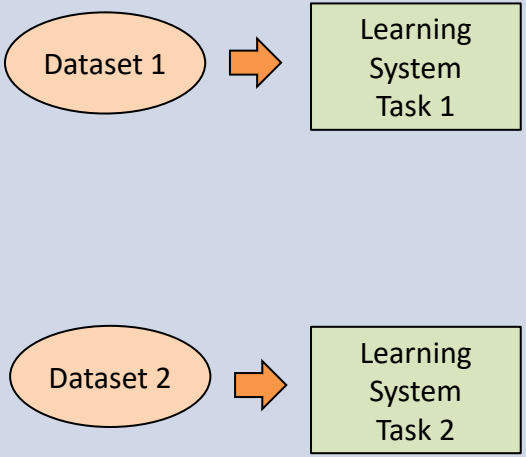
Source: Simone Bianco et al. 2018

# Transfer Learning

# Transfer learning

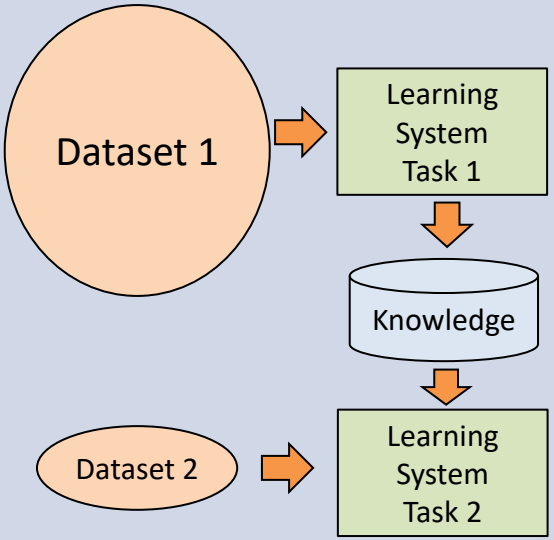
## Traditional ML

- Isolated single task learning
- Knowledge is not retained
- Easily overfit if dataset is small



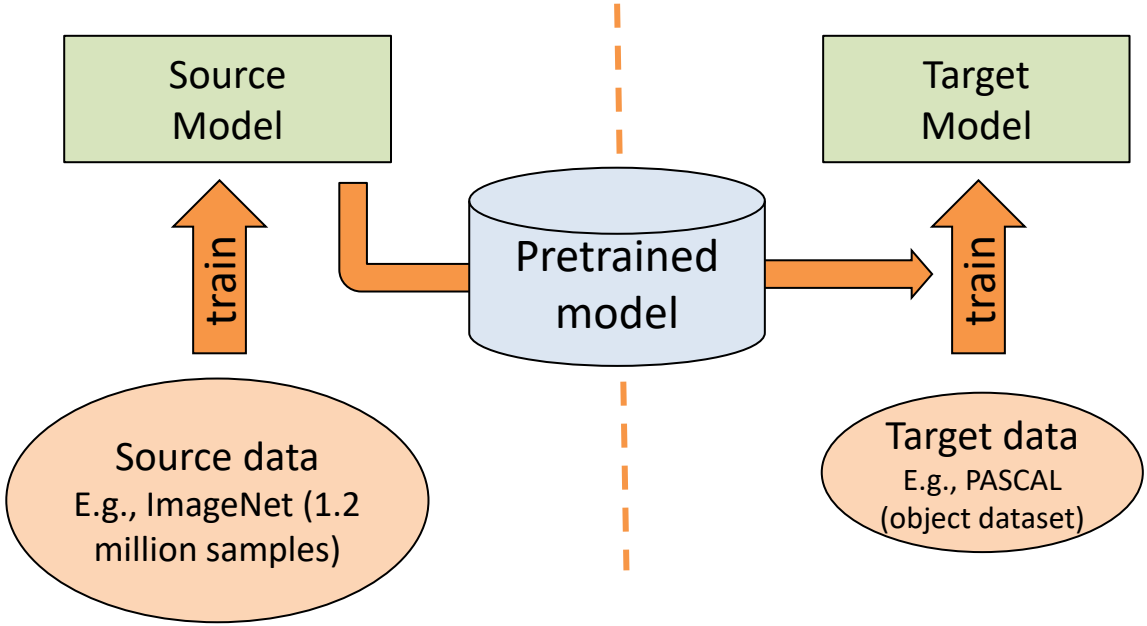

## Transfer Learning

- New task is learnt on top of previously learnt task
- Improves generalization of new task even for small training set



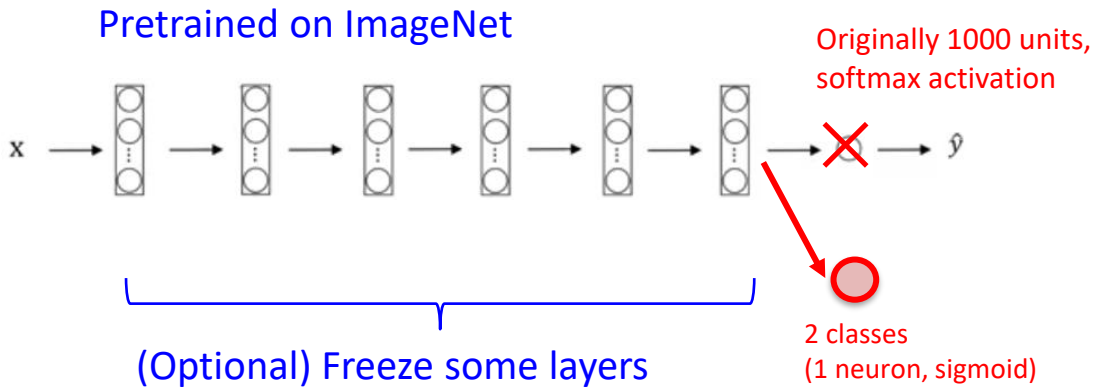
# Transfer learning

- Instead of training from scratch
  - Takes a network trained on a different domain and source task (with many annotated training samples)
  - Adapt it to your domain and targeted task



# Adapting a pre-trained model

To adapt a pre-trained model, replace the classification layer. Then, retrain the model (can opt to freeze some layers if overfitting occurs)

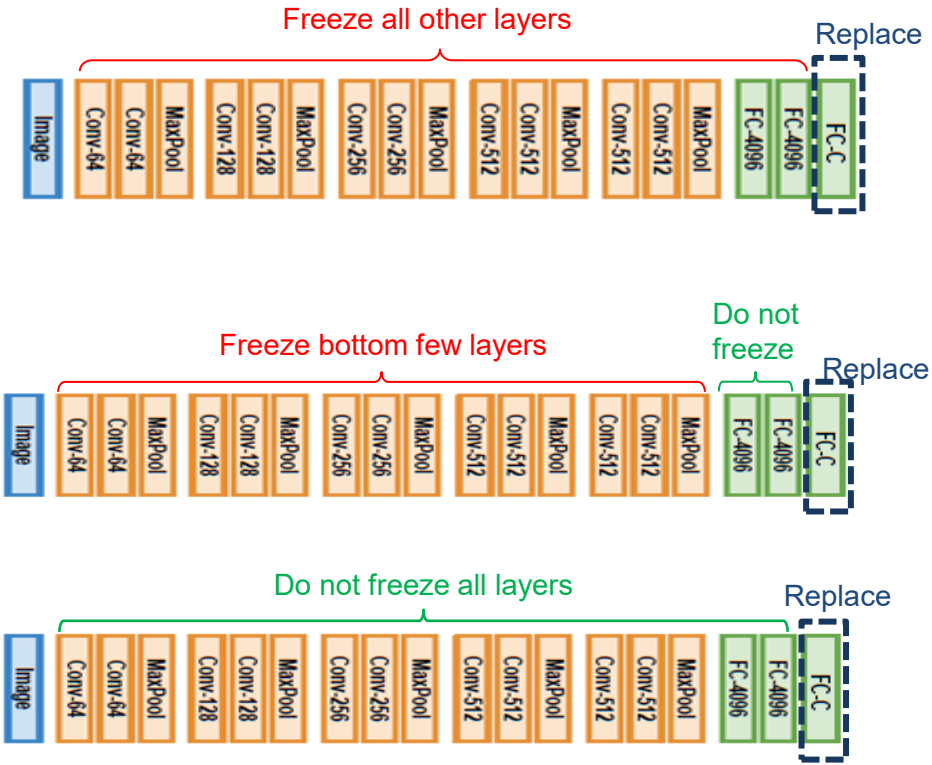



② Train the model with the targeted training set

① Replace the classification layer

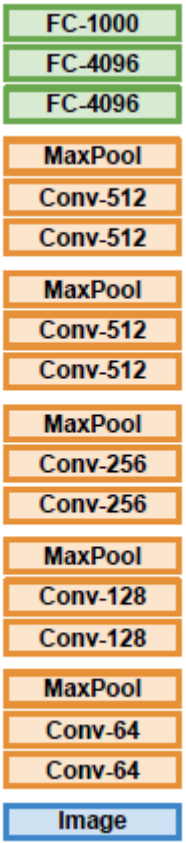
# Feature extractor vs fine-tuning

- For small dataset, use as feature extractor → reduces overfitting
- For medium dataset, finetune a few top layers (top layers are more class specific)
- For large dataset, finetune all layers




When fine-tuning, use a lower learning rate, e.g., 1/10 of original LR

# How many layers to finetune?




- Features in the deeper layers are more class specific
- Features in the shallower layers are more class agnostic (general)
- How many (top) layers to finetune?

| | **Very similar dataset** | **Very different dataset** |
|---|---|---|
| **Targeted dataset is small** | Use pre-trained model as feature extractor | You're in trouble… Try linear classifier from different stages |
| **Targeted dataset is big** | Finetune a few top layers | Finetune the whole network |

# Benefits of Transfer learning

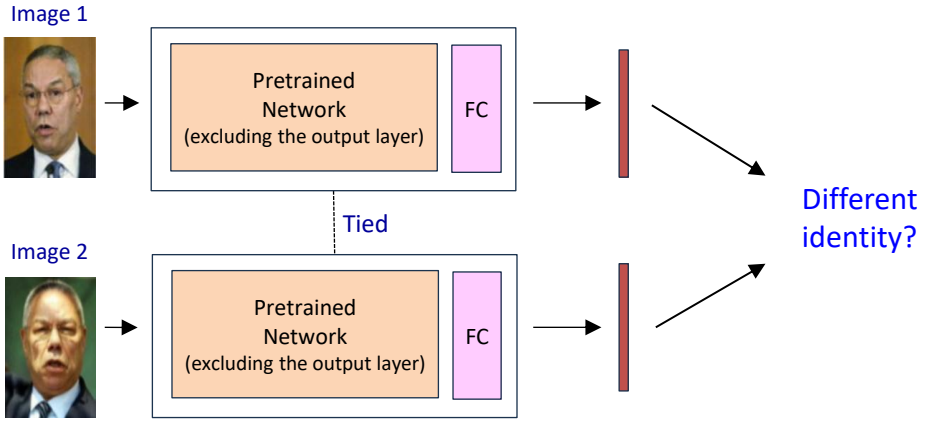
- Transfers the knowledge from one domain to another
- Training on a pre-trained model can improve the result significantly
- Allow us to achieve very good result even with a very small data set.
- Deep learning frameworks provide a "Model Zoo" of pretrained models so that you do not have to train your own.
  PyTorch: https://pytorch.org/vision/stable/models.html

# Applications: Face Recognition

# Siamese Network for Contrastive Task

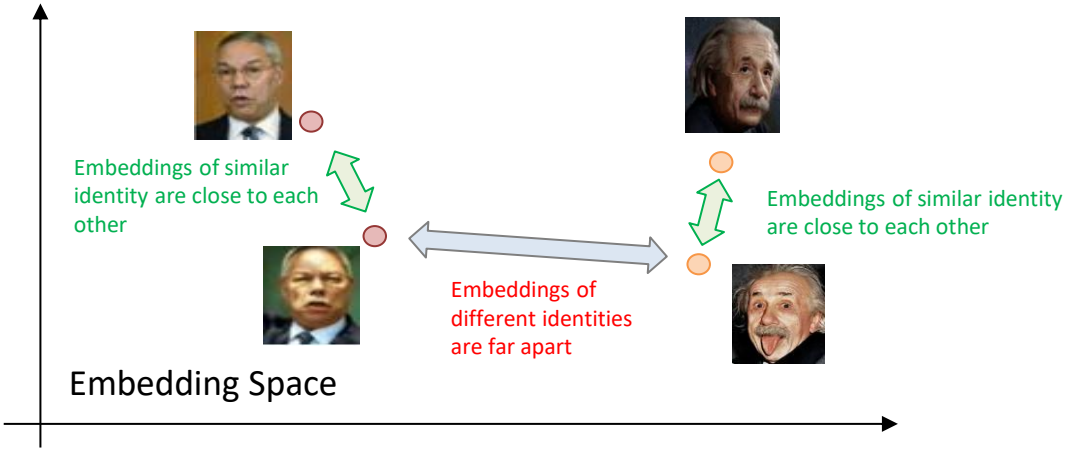
[Schroff, F., et al. *Facenet: A unified embedding for face recognition and clustering*. CVPR 2015.]

- **Goal**: Determine if two face images belong to the same identity
- **Architecture**: Two identical neural networks (shared weights) extracts the feature vector (embedding) for each image.



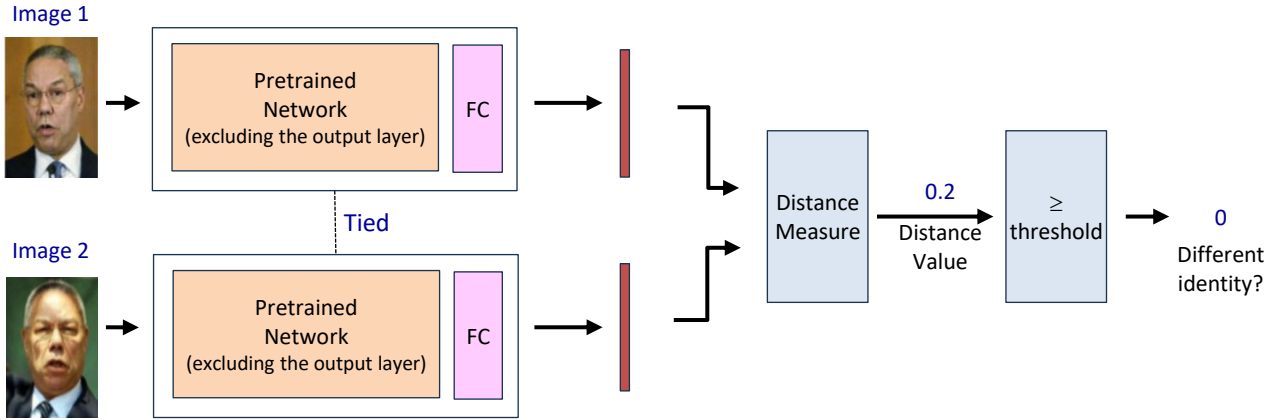

- Use a pretrained network (e.g., ResNet34) is used to extract the feature
- Use a custom FC to project the extracted feature into embedding vector.
- Compare the two embedding vectors: Different identities?

# Face recognition in the Embedding Space




- The embedding vector resides in an embedding space
- Compare the distance to determine identity
  - Small distance: same identity
  - Large distance: different identity
- Learns similarity, not classification
  - Works well with limited data
  - Easily generalizes to new identity

# Process Flow




① **Input**: two images

② Uses the twin networks to extract embeddings (e.g., 128-D vector) for both images

③ Compute distance (Euclidean or cosine distance)

④ Apply threshold
- **Small distance** → same person (label = 0)
- **Large distance** → Different person (label = 1)

# Training with Contrastive Loss

- The loss function learn embeddings where same identity are close together and different identities far apart.
- **Contrastive Loss**:

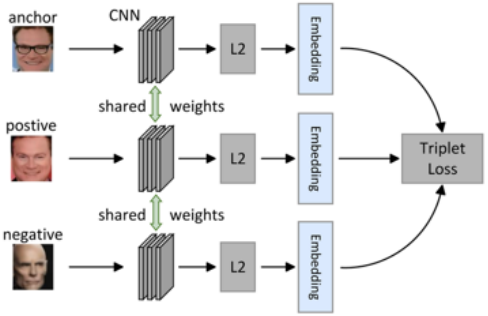
$$L = (1 - y)D^2 + y \max(0, m - D)^2$$

*D*: distance between embeddings
*y* = 0 (same person), y = 1 (different person)
*m*: margin

- The first term pulls similar pairs closer
- The second term pushes dissimilar parts apart beyond the margin.
- **Weakness**: Contrastive loss treats pairs independently, so it doesn't enforce relative ordering between similar and dissimilar samples.

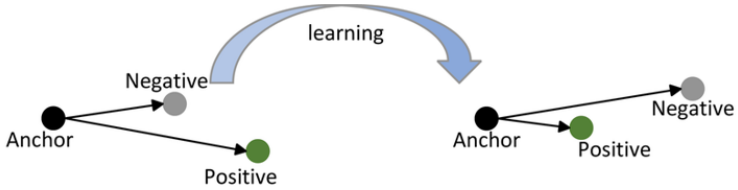
# Training with Triplet Loss

- **Triplet Loss** trains three images
- Ensures an anchor image is closer to a positive (same identity) image than to a negative (different identity) image by a margin.




$$L = \max\ (0, D(a,p) - D(a,n) + m)$$

*D*: distance between embeddings
a: anchor,
*p*: positive (same identity)
*n*: negative (different identity)
*m*: margin




- Enforces relative ordering, ensuring each anchor is closer to its positive than to any negative, producing better-separated embeddings.

# Training with Triplet Loss

- Requires careful sampling of informative
  - Compute full pair-wise distances and select hardest / semi-hard
  - Hard negative: Negative closer than the positive
  - Semi-hard negative: Father than positive but within margin
  - Easy : Father than positive and out of margin
- Use large batch size
  - At least 32-128 images
  - More negatives to choose from
  - Sample P identities x K images each (e.g., 8x4)
- Apply curriculum strategy
  - Avoid only hardest samples early → can destabilize training
  - Early training: mostly random or semi-hard
  - Later training: use more hard negatives
- Distance measure
  - Recommendation: L2-normalized embedding + cosine similarity

*Thank you*

*for your attention*